

Discesa del Gradiente

Metodi Statistici e Matematici per il Machine Learning

Traduzione italiana — materiale originale a cura di Davide Evangelista

Abbiamo visto che la **fase di training** di un modello di Machine Learning può essere espressa come un **problema di ottimizzazione**: data una funzione di perdita

$$\mathcal{L}(\Theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\Theta}(x^{(i)}), y^{(i)}),$$

il nostro obiettivo è trovare

$$\Theta^* = \arg \min_{\Theta} \mathcal{L}(\Theta).$$

Per casi semplici (come la regressione lineare con un solo parametro), questo problema di minimizzazione può ammettere una soluzione in forma chiusa. In generale, però, la funzione di perdita è **non lineare, ad alta dimensionalità, e spesso non convessa**, il che rende impossibili le soluzioni analitiche. Ci affidiamo quindi ad **algoritmi di ottimizzazione iterativi**, il più fondamentale dei quali è la **Discesa del Gradiente**.

1 Condizioni di ottimalità del primo (e secondo) ordine

Per problemi non vincolati e differenziabili $\min_{\Theta} \mathcal{L}(\Theta)$:

- **Condizione necessaria del primo ordine**: ogni minimizzatore locale Θ^* soddisfa

$$\nabla_{\Theta} \mathcal{L}(\Theta^*) = 0.$$

I punti con gradiente nullo sono chiamati **punti stazionari** (possono essere minimi, massimi, o punti di sella).

- **Test del secondo ordine** (quando $\mathcal{L}$ è due volte differenziabile): sia $H(\Theta) = \nabla_{\Theta}^2 \mathcal{L}(\Theta)$ la matrice Hessiana.
 - Se $H(\Theta^*)$ è **definita positiva**, Θ^* è un **minimo locale stretto**.
 - Se $H(\Theta^*)$ è **indefinita**, Θ^* è un **punto di sella**.
 - Se $\mathcal{L}$ è **convessa**, allora **ogni** punto stazionario è un **minimo globale**. Se è **strettamente convessa**, il minimizzatore globale è **unico**.

Nota

Figura del testo originale (non riprodotta qui): un grafico che mostra tre tipi di punto stazionario su una superficie 2D — un minimo (curvatura positiva in ogni direzione), un massimo (curvatura negativa in ogni direzione) e un punto di sella (curvatura positiva in una direzione, negativa nell'altra) — corrispondenti rispettivamente a Hessiana definita positiva, definita negativa e indefinita.

2 L'algoritmo di Discesa del Gradiente

2.1 Intuizione

Il gradiente di una funzione differenziabile $\nabla_{\Theta}\mathcal{L}(\Theta)$ punta nella direzione di **massima crescita** (steepest ascent). Se vogliamo minimizzare la funzione, dobbiamo muoverci nella **direzione opposta**. Questo porta alla regola di aggiornamento:

$$\Theta^{(k+1)} = \Theta^{(k)} - \eta_k \nabla_{\Theta}\mathcal{L}(\Theta^{(k)}),$$

dove $\eta_k > 0$, $k = 1, \dots, \text{maxit}$, è il **learning rate** (passo di aggiornamento).

L'algoritmo risultante è quindi il seguente:

Input: parametri iniziali $\Theta^{(0)}$, sequenza di learning rate $\{\eta_k\}_k$, criterio di arresto.

Ripeti finché il criterio di arresto non è soddisfatto:

1. Calcola il gradiente $\nabla_{\Theta}\mathcal{L}(\Theta^{(k)})$.
2. Aggiorna i parametri:

$$\Theta^{(k+1)} = \Theta^{(k)} - \eta_k \nabla_{\Theta}\mathcal{L}(\Theta^{(k)}).$$

Nota

L'algoritmo di discesa del gradiente è un esempio di metodo di discesa (*descent method*):

$$\Theta^{(k+1)} = \Theta^{(k)} + \eta_k p_k, \quad k = 0, 1, \dots, \text{maxit},$$

dove la direzione di discesa p_k deve essere scelta in modo che, per ogni $k \in \mathbb{N}$:

$$\nabla_{\Theta}\mathcal{L}(\Theta^{(k)})^T p_k \leq 0,$$

per garantire la convergenza a un punto stazionario. Poiché per GD si ha $p_k = -\nabla_{\Theta}\mathcal{L}(\Theta^{(k)})$, la condizione sopra è sempre soddisfatta:

$$\nabla_{\Theta}\mathcal{L}(\Theta^{(k)})^T p_k = -\nabla_{\Theta}\mathcal{L}(\Theta^{(k)})^T \nabla_{\Theta}\mathcal{L}(\Theta^{(k)}) = -\|\nabla_{\Theta}\mathcal{L}(\Theta^{(k)})\|_2^2 \leq 0.$$

Pertanto, l'algoritmo GD converge **sempre** a un punto stazionario nel limite di iterazioni infinite.

Attenzione

Non bisogna confondere questa Discesa del Gradiente “full-batch” con la **Discesa del Gradiente Stocastica** (SGD), che verrà introdotta nel prossimo capitolo.

3 Selezione dei parametri

3.1 Scelta dell'iterata iniziale

L'algoritmo di Discesa del Gradiente (GD) richiede all'utente di fornire un'iterata iniziale $\Theta^{(0)} \in \mathbb{R}^n$. Teoricamente, poiché GD ha una proprietà di *convergenza globale*, per qualsiasi $\Theta^{(0)}$ convergerà sempre a un **punto stazionario** di $\mathcal{L}(\Theta)$, cioè a un punto tale che $\nabla_{\Theta}\mathcal{L}(\Theta) = 0$.

Se $\mathcal{L}(\Theta)$ è convessa, allora ogni punto stazionario è un minimo (globale) di $\mathcal{L}(\Theta)$, il che implica che la scelta di $\Theta^{(0)}$ non è particolarmente importante, e possiamo sempre impostare $\Theta^{(0)} = 0$. D'altra parte, quando $\mathcal{L}(\Theta)$ non è convessa, dobbiamo scegliere $\Theta^{(0)}$ il più vicino possibile al punto stazionario “giusto”, per aumentare le probabilità di raggiungerlo. Se non è disponibile una stima del punto di minimo corretto, considereremo semplicemente $\Theta^{(0)} = 0$ per arrivare a un minimo locale generico.

3.2 Selezione del passo (Learning Rate)

La scelta di $\{\eta_k\}_k$ influisce in modo critico sulla convergenza.

- **Passo costante:** dato un valore η , si pone $\eta_k = \eta \ \forall k = 1, \dots, \text{maxit}$. Semplice, ma dipendente dalla scala del problema. In particolare:
 - Se η è **troppo piccolo**: GD non converge, poiché le iterate si bloccano prima di raggiungere la convergenza;
 - Se η è **troppo grande**: le iterate rimbalzano avanti e indietro attorno al minimo, senza riuscire a raggiungerlo;
 - Se η è **ben calibrato**: il metodo GD converge a un punto stazionario, con una velocità proporzionale a quanto è grande η .

Nota

Figura del testo originale (non riprodotta qui): tre traiettorie di GD su un paesaggio a ellissi che partono dallo stesso punto iniziale — con η troppo piccolo la traiettoria avanza a piccoli passi senza raggiungere il minimo entro le iterazioni concesse; con η troppo grande la traiettoria oscilla in modo instabile attorno al minimo senza convergere; con η ben calibrato la traiettoria scende in modo regolare fino al minimo.

- **Backtracking line search:** per ogni $k \in 1, \dots, \text{maxit}$, si pone $\eta_k = \bar{\eta}$ e lo si riduce di un fattore $\beta \in (0, 1)$ finché non è soddisfatta la condizione di **Armijo**:

$$\mathcal{L}(\Theta^{(k)} - \eta g^{(k)}) \leq \mathcal{L}(\Theta^{(k)}) - c \eta_k \|g^{(k)}\|^2, \quad g^{(k)} = \nabla_{\Theta} \mathcal{L}(\Theta^{(k)}), \quad c \in (0, 1).$$

Backtracking:

```
import numpy as np

def backtracking(L, grad_L, theta, eta0=1.0, beta=0.5, c=1e-4):
    """
    Restituisce un passo eta che soddisfa la condizione di Armijo:
    L(theta - eta*g) <= L(theta) - c * eta * ||g||^2
    Input:
    - L: R^n -> R
    - grad_L: R^n -> R^d
    - theta: punto corrente (np.ndarray)
    - eta0: passo iniziale
    - beta: fattore di riduzione in (0,1)
    - c: costante di Armijo in (0,1)
    """
    eta = eta0
    g = grad_L(theta)
    g_norm2 = np.dot(g, g)
    while L(theta - eta * g) > L(theta) - c * eta * g_norm2:
        eta *= beta
    return eta
```

3.3 Criteri di arresto

In pratica, non possiamo iterare all'infinito. I **criteri di arresto** più comuni includono:

- La **norma del gradiente** è piccola:

$$\|\nabla_{\Theta} \mathcal{L}(\Theta^{(k)})\| < \text{tol}_{\mathcal{L}},$$

il che significa che siamo vicini a un punto stazionario.

- La **variazione della loss** è trascurabile:

$$|\mathcal{L}(\Theta^{(k+1)}) - \mathcal{L}(\Theta^{(k)})| < \text{tol}_{ES}.$$

Chiamato **early stopping** nella pratica ML (di solito stimato sul validation set).

- L'**aggiornamento dei parametri** è piccolo:

$$\|\Theta^{(k+1)} - \Theta^{(k)}\| < \text{tol}_{\Theta}.$$

- Viene raggiunto un **numero massimo di iterazioni** maxit.

4 Malcondizionamento e geometria

Il malcondizionamento (*ill-conditioning*) si manifesta come insiemi di livello allungati e traiettorie di GD a **zig-zag**, che rendono difficile per GD raggiungere la convergenza.

Dato $0 < \lambda_1 \ll \lambda_2$, consideriamo la funzione di perdita:

$$\mathcal{L}(\Theta) = \frac{1}{2} \lambda_1 \Theta_1^2 + \lambda_2 \Theta_2^2,$$

Gli insiemi di livello sono ellissi i cui assi scalano con $\lambda_1^{-1/2}$ e $\lambda_2^{-1/2}$. Quando il rapporto λ_2/λ_1 è grande, la Discesa del Gradiente con η fisso progredisce lentamente e procede a “zig-zag” attraverso valli strette.

Nota

Esercizio (intuizione sul numero di condizionamento; moltiplicazione a sinistra): disegna gli insiemi di livello di $\mathcal{L}(\Theta) = \frac{1}{2} \Theta^T A \Theta$ per $A = \text{diag}(\lambda_1, \lambda_2)$.

Funzione di supporto per il grafico degli insiemi di livello:

```
import numpy as np
import matplotlib.pyplot as plt

def quad_levelsets(A, xlim=(-3,3), ylim=(-3,3), ngrid=400, ncontours=12, title=None):
    xs = np.linspace(xlim[0], xlim[1], ngrid)
    ys = np.linspace(ylim[0], ylim[1], ngrid)
    X, Y = np.meshgrid(xs, ys)
    Z = 0.5*(A[0,0]*X**2 + 2*A[0,1]*X*Y + A[1,1]*Y**2) # theta^T A theta
    cs = plt.contour(X, Y, Z, levels=ncontours)
    plt.clabel(cs, inline=True, fontsize=8)
    plt.axhline(0, lw=0.5, color='k')
    plt.axvline(0, lw=0.5, color='k')
    plt.gca().set_aspect('equal', 'box')
    if title:
        plt.title(title)
    plt.xlabel(r'$\theta_1$')
    plt.ylabel(r'$\theta_2$')
    plt.grid(alpha=0.2)
    plt.show()

# Esempio: ellisse mal condizionata
A = np.diag([1.0, 25.0]) # lambda1 << lambda2
quad_levelsets(A, title='Insiemi di livello di 0.5 * theta^T A theta')
```

5 Limiti della Discesa del Gradiente pura

Nonostante la sua utilità, la Discesa del Gradiente presenta diversi svantaggi:

1. **Costo computazionale elevato:** calcolare il gradiente richiede di scandire l'intero dataset ad ogni passo.
2. **Sensibilità al learning rate:** troppo piccolo $\Rightarrow$ convergenza lenta, troppo grande $\Rightarrow$ divergenza/oscillazioni.
3. **Minimi locali, punti di sella e plateau** in paesaggi non convessi.
4. Il **malcondizionamento** causa progressi lenti e zig-zag.

5.1 Motivazione per la Discesa del Gradiente Stocastica

Questi limiti motivano lo sviluppo di algoritmi più pratici. In particolare, invece di calcolare il gradiente usando *l'intero dataset* ad ogni iterazione, è possibile approssimarlo usando un **sottoinsieme dei dati (mini-batch)**. Questo dà origine alla **Discesa del Gradiente Stocastica (SGD)**, argomento del prossimo capitolo.

SGD affronta il problema della scalabilità e spesso migliora la generalizzazione, al costo di introdurre stocasticità negli aggiornamenti.

In sintesi, la **Discesa del Gradiente** fornisce i principi fondamentali dell'ottimizzazione nel Machine Learning:

- È un metodo iterativo per minimizzare la loss basato sui **gradienti negativi**.
- Il suo comportamento dipende da **convessità/convessità stretta**, **regolarità (smoothness)**, **scelta del passo** (costante, ricerca esatta, o **backtracking/Armijo**), e **criteri di arresto**.
- I suoi limiti pratici portano naturalmente alla **Discesa del Gradiente Stocastica**, che studieremo nel prossimo capitolo.

*Traduzione italiana del materiale del corso “Statistical and Mathematical Methods for Machine Learning”
(pagina originale in inglese).*

Materiale originale di Davide Evangelista — © 2023.